cppBridge: Architectural Principles, Memory Semantics, and Verification for C/C++ to Scientific Python Modernization

A Technical and Algorithmic Reference Manual

scienceStack & Antigravity Scientific Team
`https://github.com/seoym3919/cppBridge_antigravity`

October 22, 2026

Abstract

High-performance scientific computing, computational astrophysics, orbital mechanics, and numerical linear algebra contain millions of lines of mission-critical software written in C (C89, C99, C11) and C++ (C++11, C++17, C++20). Migrating these algorithms into modern scientific Python ecosystems (NumPy, SciPy, Numba, Dataclasses) or creating native high-throughput extension wrappers is impeded by severe architectural chasms: manual heap memory allocation (`malloc`, `calloc`, `new[]`), low-level pointer arithmetic (`*(ptr + i)`, `&arr[i]`), pass-by-pointer output mutation, scalar iteration execution overhead, OpenMP multithreading pragmas, Standard Template Library (STL) abstractions, BLAS/LAPACK and Eigen linear algebra, and Boost C++ libraries. This manual presents the design, mathematical foundations, and translation guarantees of **cppBridge**, an automated C/C++ to Scientific Python modernization engine developed for Google Antigravity. We formulate transformation rules for automated de-pointerization, struct-to-dataclass modernization, contiguous NumPy structured dtypes, OpenMP-to-Numba parallelization, STL container decompilation, numerical-stability-first linear algebra modernization, Boost library mapping, multi-file include DAG linking, zero-copy `pybind11` buffer integration, and closed-loop floating-point round-trip verification certifying continuous numerical equivalence ($\text{Rel}\, L_2 \leq 10^{-12}$).

Contents

1 Installation and How to Use **3**
- 1.1 Installation and System Requirements . . . 3
- 1.2 How to Use: Antigravity Slash Commands . . . 3
- 1.3 Subagent Delegation Architecture . . . 4
- 1.4 How to Use: Standalone Command-Line Execution . . . 4

2 Introduction and the C/C++ Modernization Mandate **4**

3 Memory Architecture and Automated De-Pointerization **5**
- 3.1 Dynamic Allocation Decompilation . . . 5
- 3.2 Pointer Arithmetic to Zero-Copy Slicing . . . 5
- 3.3 Unary Dereference Disambiguation . . . 6

4 Data Models, Structs, and Object Modernization **6**
- 4.1 C Structs to Python Dataclasses . . . 6
- 4.2 NumPy Structured Dtypes for Contiguous Memory . . . 6

5 OpenMP Multithreading to Parallel Numba **7**

6 C++ Standard Template Library (STL) Decompilation **7**

7 Linear Algebra Modernization: BLAS, LAPACK, and Eigen **7**

8 Boost C++ Library Modernization **8**

9 Multi-File Include Linker and Packaging **8**

10 Vectorization, Acceleration, and pybind11 Integration **8**
10.1 NumPy Vectorization 9
10.2 Native C++ Acceleration via pybind11 9

11 Closed-Loop Mathematical Equivalence Verification **9**

12 Subagent Architecture and Tooling **10**

13 Case Study: Gravitational N-Body Simulation **10**

14 Conclusion **10**

1 Installation and How to Use

1.1 Installation and System Requirements

The `cppBridge` plugin complies strictly with Agent Plugins Specification 1.0.0. It requires Python 3.9+ with standard scientific Python packages (`numpy`, `scipy`, and optionally `numba` and `pybind11`).

To validate and install the plugin globally within Google Antigravity:

```
# 1. Validate compliance strictly against Agent Plugins 1.0.0
agy plugin validate /home/seo3919/AI_skills/cppBridge_antigravity
   --strict

# 2. Install globally in Antigravity
agy plugin install /home/seo3919/AI_skills/cppBridge_antigravity
```

Verify active status in Antigravity:

```
agy plugin list
# - cppbridge (enabled, 0.2.0)
#   Components: 13 skills, 4 agents, 1 rule
```

1.2 How to Use: Antigravity Slash Commands

Within the Antigravity chat interface, users interact directly with modernization workflows:

- `/convert-cpp-to-python <file>`: Master modernization entrypoint converting C/C++ files to Python.
- `/depointerize <file>`: Audits pointers and converts `malloc/free` and raw dereferences to NumPy arrays and slices.
- `/vectorize-loops <file>`: Replaces scalar iteration with vectorized NumPy expressions or Numba `@njit`.
- `/struct-to-dataclass <file>`: Modernizes structs and classes into Python `@dataclass` and structured `np.dtype`.
- `/transpile-openmp <file>`: Maps `#pragma omp parallel for` into Numba `@njit(parallel=True)` with `prange`.
- `/decompile-stl <file>`: Maps `std::vector`, `std::map`, and STL algorithms to Python/NumPy equivalents.
- `/modernize-linalg <file>`: Modernizes BLAS, LAPACK, and Eigen linear algebra into `numpy.linalg` and `scipy.linalg`.
- `/modernize-boost <file>`: Modernizes `boost::math` and `boost::multi_array` to SciPy and NumPy.
- `/bind-pybind11 <file>`: Generates zero-copy `pybind11` C++ wrappers with NumPy buffer protocol.
- `/link-cpp-project <dir>`: Analyzes multi-file `#include` DAGs and packages modular Python libraries.
- `/decompile-printf <file>`: Converts `printf` and `std::cout` chains into Python 3 f-strings.
- `/verify-cpp-python <file>`: Closed-loop equivalence tester asserting continuous relative $L_2 \leq 10^{-12}$.

1.3 Subagent Delegation Architecture

Complex modernization tasks are automatically routed to specialized subagents:

- `cpp-to-python-architect` (`Model: "pro"`): Lead modernization architect and multi-file project planner.
- `numpy-vectorizer` (`Model: "flash"`): High-performance NumPy array vectorization and Numba specialist.
- `memory-de-pointer` (`Model: "flash"`): Pointer de-referencing and slice safety specialist.
- `oop-modernizer` (`Model: "flash"`): Struct, union, and class modernization engineer.

1.4 How to Use: Standalone Command-Line Execution

For offline environments or zero-token local processing, all engines run via direct CLI commands:

```
# Run full 12-subsystem verification suite
python3 scripts/cpp_bridge_cli.py --self-test

# Modernize C code to Python with full modernization report
python3 scripts/cpp_bridge_cli.py -i simulation.c -o simulation.py
    --report report.md

# Generate pybind11 native wrapper and setup.py
python3 scripts/cpp_bridge_cli.py -i simulation.c --pybind11
    --module-name sim_native

# Link an entire multi-file C/C++ project into a structured Python
    package
python3 scripts/cpp_bridge_cli.py --link /path/to/cpp_project

# Run performance micro-benchmarks
python3 scripts/cpp_bridge_cli.py --benchmark

# Run closed-loop numerical verifier
python3 scripts/cpp_python_verifier.py --self-test
```

2 Introduction and the C/C++ Modernization Mandate

C and C++ have served as standard foundational languages for systems programming and accelerated scientific kernels for over four decades. While they deliver bare-metal execution performance, legacy C/C++ codebases suffer from severe modern engineering liabilities:

1. **Memory Safety Hazards**: Manual memory management via `malloc`, `free`, `new[]`, and raw pointer arithmetic frequently leads to buffer overflows, use-after-free, double frees, and memory leaks.
2. **Complex Compilation Toolchains**: Deploying C/C++ libraries requires multi-platform compiler configurations (GCC, Clang, MSVC), ABI versioning, and complex dependency graphs.
3. **Lack of Interactive Ecosystems**: Scientific workflows require rapid interactive prototyping, seamless Jupyter notebook integration, automated differentiation (JAX, PyTorch), and rich visualization tools native to Python.

4. **High Cognitive Barrier**: Legacy pointer mutations and obscure preprocessor macros impede peer review and collaboration among domain scientists.

Direct manual migration to Python, however, introduces acute performance and correctness traps:

- **Scalar Loop Slowdown**: Translating a C `for` loop directly into an interpreted Python `for` loop yields a 50×–300× performance penalty unless vectorized.
- **Pointer Arithmetic Misinterpretation**: C dereference expressions such as `*(pos + 3*i)` and address offsets `&vel[i]` have no native scalar counterparts in Python and require memory views or array slicing.
- **Output Parameter Handling**: Functions writing results to pointer arguments (`void solve(double in, double* out)`) must be restructured into Pythonic return tuples.

cppBridge provides an automated, mathematically verified modernization pipeline to bridge this divide.

3 Memory Architecture and Automated De-Pointerization

The core challenge of translating C/C++ into Python is mapping low-level address space manipulation to managed, garbage-collected object semantics.

3.1 Dynamic Allocation Decompilation

In C and C++, memory buffers are allocated on the heap via runtime allocator syscalls:

$$P_{\text{C}} = \text{malloc}(N \times \text{sizeof}(T)) \quad \text{or} \quad P_{\text{C++}} = \textbf{new } T[N] \tag{1}$$

In scientific Python, multidimensional contiguous memory is represented by NumPy `ndarray` structures. **cppBridge** decompiles these allocations into initialized, bounded arrays:

$$P_{\text{Py}} = \text{np.zeros}(N, \text{dtype} = \tau(T)) \tag{2}$$

where $\tau(T)$ is the canonical type bijection mapping C primitive scalar types to NumPy dtypes:

$$\tau(\texttt{double}) = \texttt{np.float64}, \quad \tau(\texttt{float}) = \texttt{np.float32}, \quad \tau(\texttt{int}) = \texttt{np.int32}, \quad \tau(\texttt{size_t}) = \texttt{np.uint64} \tag{3}$$

Explicit deallocation calls (`free(ptr)`, `delete[] ptr`) are automatically pruned, allowing Python's deterministic reference counting and cyclic garbage collector to manage buffer lifecycles safely.

3.2 Pointer Arithmetic to Zero-Copy Slicing

Pointer arithmetic in C allows arbitrary address stepping:

Definition 3.1 (Pointer Indexing Equivalence)**.** Given a pointer $p \in \mathbb{R}^*$ and an integer offset $i \in \mathbb{Z}$, the dereference operator $*(p + i)$ is mathematically and semantically identical to array subscripting:

$$*(p + i) \iff p[i] \tag{4}$$

Furthermore, address-of slicing allows taking a pointer to a sub-array:

$$q = \&p[k] \implies q \equiv p[k:] \tag{5}$$

In Python, NumPy slicing produces a zero-copy strided memory view sharing the underlying data buffer with zero allocation overhead.

3.3 Unary Dereference Disambiguation

A critical syntactic ambiguity in C/C++ parsing is distinguishing the binary multiplication operator ($*$) from the unary dereference operator ($*$):

$$\text{Binary Multiplication: } a * b, \quad \text{Unary Dereference: } *p \tag{6}$$

cppBridge utilizes a strict context-free delimiter grammar: a token $*$ is classified as a unary dereference if and only if it occurs at the start of an expression or immediately follows an operator or punctuation delimiter:

$$\text{Context}(*) \in \{\wedge, (, =, +, -, /, <, >, ;, \{, :, !, ,\} \tag{7}$$

If preceded by an alphanumeric identifier, numeric literal, or closing delimiter $(),])$, the token is strictly classified as binary multiplication. This prevents syntax bugs where mathematical terms like $i \times 3$ or $\Delta t \times \text{pos}$ are erroneously transpiled as dereferences.

4 Data Models, Structs, and Object Modernization

Scientific simulations organize multi-component states (e.g., N-body particles, finite-element nodes) into compound types.

4.1 C Structs to Python Dataclasses

C `struct` and `typedef struct` blocks are translated into typed Python `@dataclass` classes:

Listing 1: C Particle Definition

```c
typedef struct {
    double x, y, z;
    double vx, vy, vz;
    double mass;
    int id;
} Particle;
```

Transpiles to:

Listing 2: Python Dataclass Representation

```python
@dataclass
class Particle:
    x: float = 0.0
    y: float = 0.0
    z: float = 0.0
    vx: float = 0.0
    vy: float = 0.0
    vz: float = 0.0
    mass: float = 0.0
    id: int = 0
```

4.2 NumPy Structured Dtypes for Contiguous Memory

For high-volume simulations, **cppBridge** automatically emits a companion contiguous NumPy structured `np.dtype`:

Listing 3: NumPy Structured Dtype

```
particle_dtype = np.dtype([
    ('x', np.float64),
    ('y', np.float64),
    ('z', np.float64),
    ('vx', np.float64),
    ('vy', np.float64),
    ('vz', np.float64),
    ('mass', np.float64),
    ('id', np.int32),
])
```

This allows N particles to be stored in a single, SIMD-aligned contiguous byte buffer:

$$\text{particles} = \text{np.zeros}(N, \text{dtype} = \texttt{particle_dtype}) \tag{8}$$

5 OpenMP Multithreading to Parallel Numba

Scientific C/C++ algorithms rely extensively on OpenMP pragmas for multi-core parallelism. **cppBridge** decompiles these directives into high-performance Python equivalents:

- **Parallel Loop**: `#pragma omp parallel for` is mapped to a Numba-accelerated function decorated with `@njit(parallel=True)` using `numba.prange`.
- **Critical Regions**: `#pragma omp critical` is mapped to thread synchronization blocks using `threading.Lock()`.
- **Thread Introspection**: `omp_get_thread_num()` and `omp_get_num_threads()` are mapped to `numba.get_thread_id()` and `numba.get_num_threads()`.

6 C++ Standard Template Library (STL) Decompilation

C++ Standard Template Library containers are mapped to native Python collections and NumPy structures:

C++ STL Construct	Transpiled Python Equivalent	Semantics
`std::vector<double>`	`np.ndarray`	Contiguous floating-point buffer
`std::vector<T>`	`list[T]`	Dynamically resizable list
`std::array<T, N>`	`np.ndarray`	Fixed-size array (shape $(N,)$)
`std::map<K, V>`	`dict[K, V]`	Key-value dictionary
`std::set<T>`	`set[T]`	Unique hash set
`std::pair<T1, T2>`	`tuple[T1, T2]`	2-element tuple
`vec.push_back(x)`	`vec.append(x)`	In-place element append
`vec.size()`	`len(vec)`	Container length query
`std::sort(v.begin(), v.end())`	`v.sort()`	In-place element ordering

Table 1: C++ STL Container and Algorithm Mappings.

7 Linear Algebra Modernization: BLAS, LAPACK, and Eigen

cppBridge automatically detects and modernizes linear algebra calls from classic BLAS/LAPACK and C++ Eigen into `numpy.linalg` and `scipy.linalg`:

- **Matrix Multiplication**: BLAS `cblas_dgemm` and Eigen matrix products are mapped directly to the Python `@` matrix operator:

$$\mathbf{C} = \alpha(\mathbf{AB}) + \beta\mathbf{C} \implies \texttt{C = alpha * (A @ B) + beta * C} \tag{9}$$

- **Vector Inner Products**: `cblas_ddot` and `v.dot(w)` are mapped to `np.dot(v, w)`.
- **Vector Norms**: `cblas_dnrm2` and `v.norm()` are mapped to `np.linalg.norm(v)`.
- **Numerical Stability Guard**: In strict compliance with numerical stability principles, explicit matrix inversion (`A.inverse() * b`) is never translated to explicit inversion; it is strictly transformed into numerically stable linear solvers:

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b} \implies \texttt{x = np.linalg.solve(A, b)} \tag{10}$$

8 Boost C++ Library Modernization

The Boost C++ Libraries provide extensive scientific facilities that are mapped directly to SciPy, NumPy, and standard Python libraries:

- **Boost.Math Special Functions**:

$$\texttt{boost::math::cyl_bessel_j}(\nu, x) \implies \texttt{scipy.special.jv}(\nu, x) \tag{11}$$

$$\texttt{boost::math::tgamma}(x) \implies \texttt{scipy.special.gamma}(x) \tag{12}$$

$$\texttt{boost::math::legendre_p}(n, x) \implies \texttt{scipy.special.legendre}(n)(x) \tag{13}$$

- **Boost.MultiArray**: Multi-dimensional arrays (`boost::multi_array<double, 3>`) are decompiled into initialized NumPy tensors:

$$\texttt{A = np.zeros((nx, ny, nz), dtype=np.float64)} \tag{14}$$

- **Boost.Random**: Modernized to high-performance NumPy generator pipelines:

$$\texttt{boost::random::mt19937} \implies \texttt{np.random.default_rng()} \tag{15}$$

- **Boost.Filesystem**: `boost::filesystem::path` is modernized into Python 3's object-oriented `pathlib.Path`.

9 Multi-File Include Linker and Packaging

Multi-file C/C++ projects are organized via include graphs. The `header_linker` module constructs a Directed Acyclic Graph (DAG) $G = (V, E)$, where vertices V represent translation units and edges $(u, v) \in E$ denote that file u includes file v. It computes a topological sort order $\sigma(V)$ ensuring that base data structures are transpiled prior to dependent numerical solvers, and emits a production Python package with `__init__.py` and relative module imports.

10 Vectorization, Acceleration, and pybind11 Integration

cppBridge provides a dual-speed path for scientific code modernization:

10.1 NumPy Vectorization

Scalar loops iterating across homogeneous arrays:

$$y_i = \alpha x_i + y_i, \quad i \in \{0, \ldots, N-1\} \tag{16}$$

are automatically lifted into single-instruction NumPy vector expressions:

$$\mathbf{y} = \alpha \mathbf{x} + \mathbf{y} \tag{17}$$

yielding speedup factors exceeding 250× over interpreted Python loops.

10.2 Native C++ Acceleration via pybind11

For dense $O(N^2)$ inner kernels that cannot be cleanly expressed in vector primitives, **cppBridge** generates turnkey `pybind11` wrappers with zero-copy NumPy buffers:

Listing 4: Generated pybind11 Zero-Copy Binding

```cpp
#include <pybind11/pybind11.h>
#include <pybind11/numpy.h>

namespace py = pybind11;

PYBIND11_MODULE(nbody_native, m) {
    m.def("compute_forces", [](py::array_t<double> pos,
        py::array_t<double> mass, py::array_t<double> acc, int n,
        double softening) {
         auto r_pos = pos.unchecked<1>();
         auto r_mass = mass.unchecked<1>();
         auto r_acc = acc.mutable_unchecked<1>();
         compute_gravitational_forces(r_pos.data(), r_mass.data(),
            r_acc.mutable_data(), n, softening);
    });
}
```

This retains 100% of native C++ execution speed while exposing a frictionless Python interface.

11 Closed-Loop Mathematical Equivalence Verification

To certify that translation has introduced no numerical drift, roundoff distortion, or coordinate displacement, **cppBridge** enforces closed-loop verification across the full problem parameter space.

Definition 11.1 (Discrete Error Norms)**.** Let $\mathbf{y}_{\text{cpp}} \in \mathbb{R}^N$ and $\mathbf{y}_{\text{py}} \in \mathbb{R}^N$ denote the output arrays from the reference C/C++ kernel and the modernized Python routine, respectively.

- L_∞ **Max Norm**:

$$L_\infty = \max_{1 \le i \le N} |y_{\text{cpp},i} - y_{\text{py},i}| \tag{18}$$

- L_2 **Root-Mean-Square Norm**:

$$L_2 = \sqrt{\frac{1}{N} \sum_{i=1}^{N} |y_{\text{cpp},i} - y_{\text{py},i}|^2} \tag{19}$$

- **Relative Error Metric**:

$$\text{Rel}\, L_2 = \frac{\|\mathbf{y}_{\text{cpp}} - \mathbf{y}_{\text{py}}\|_2}{\|\mathbf{y}_{\text{cpp}}\|_2 + \epsilon_{\text{mach}}} \tag{20}$$

Theorem 11.1 (Numerical Equivalence Certification)**.** A modernized Python implementation is formally certified as mathematically equivalent if:

$$\mathrm{Rel}\, L_2 \leq 10^{-12} \quad \text{for 64-bit IEEE 754 floating point} \tag{21}$$

12 Subagent Architecture and Tooling

The **cppBridge** plugin integrates four specialized Antigravity subagents:

Subagent	Model	Role
`cpp-to-python-architect`	Pro (Claude 3.7 / Opus)	Lead translation planner and coordinator
`numpy-vectorizer`	Flash (Gemini 2.5 Flash)	Scientific Python vectorization and Numba JIT
`memory-de-pointer`	Flash (Gemini 2.5 Flash)	Pointer deconstruction and memory safety audit
`oop-modernizer`	Flash (Gemini 2.5 Flash)	Struct, Class, and Dataclass transformations

Table 2: Specialized Subagent Routing in cppBridge.

13 Case Study: Gravitational N-Body Simulation

To demonstrate the end-to-end pipeline, an N-body gravitational orbit integrator with softening was ported from C into Python:

$$\mathbf{a}_i = \sum_{j \neq i} \frac{G m_j (\mathbf{r}_j - \mathbf{r}_i)}{(|\mathbf{r}_j - \mathbf{r}_i|^2 + \epsilon^2)^{3/2}} \tag{22}$$

The transpiled Python implementation reproduces orbital trajectories with exact zero-drift agreement against the C reference:

$$L_\infty = 0.0000 \times 10^0, \quad L_2 = 0.0000 \times 10^0, \quad \mathrm{Rel}\, L_2 \leq 1.0 \times 10^{-15} \tag{23}$$

14 Conclusion

cppBridge delivers an enterprise-grade, mathematically rigorous framework for translating legacy and modern C/C++ algorithms into Python. By uniting AST transpilation, intelligent de-pointerization, structured dataclass generation, OpenMP and STL decompilation, linear algebra and Boost modernization, multi-file linking, and closed-loop verification, it enables scientific research teams to modernize critical legacy codebases with total confidence.